

Depicting Software Evolution Using Spirals

Phillip Entin
REVEAL @ Software Institute
USI, Lugano, Switzerland

Marco Raglianti
REVEAL @ Software Institute
USI, Lugano, Switzerland

Michele Lanza
REVEAL @ Software Institute
USI, Lugano, Switzerland

Abstract—Representing and understanding software evolution is inherently difficult, especially when dealing with long-lived software systems, due to the sheer amount of information produced during a project’s lifetime. Commits containing source code changes and branches tracking parallel development accumulate over time, creating the need for scalable, informative representations. Visualizing the evolution of large-scale software projects has two conflicting goals: Either a visualization is representing a software’s lifetime in detail, or it compresses time to provide a compact overview. As a consequence, detailed approaches have scalability issues, while more compact visualizations are hiding details. Moreover, many time-based visualizations fall short when it comes to conveying the cyclical nature of software development, which is based on humane time periods (years, months) or on project-specific time scales (release cycles).

We present an approach that maps software repositories onto spirals, with all their commits and branches. Revolutions and sectors of the spiral map onto “humane” time intervals, and can be customized to correspond to arbitrary periods. This provides human-readable time scales that reflect the underlying development processes, giving valuable insights. To exemplify the approach we built NAUTILUS, an interactive web-based tool capable of representing up to hundreds of thousands of commits and thousands of branches. We illustrate our approach on a set of repositories and discuss the emerging evolutionary patterns.

Index Terms—spiral layout, software visualization, software evolution, software repository mining, Nautilus

I. INTRODUCTION

Software evolves [1]. Over time, a rich history forms, comprising information about *who* did *what* and *when*. Thanks to modern versioning systems [2], this manifests itself as millions of changes, captured by hundreds of thousands of commits across hundreds of branches. To complicate the scenario, the atomic unit of versioning, files, can last many years or even decades. The amount of evolutionary information thus accumulated can indeed be staggering. For example, in the case of a project like the Linux kernel [3], [4] we find nearly 1.5 Million commits by more than 40k authors spanning multiple decades.¹ Recovering the data stored in a repository is a challenge by itself [5], depicting the data even more so.

Over the years, the software visualization community has greatly expanded beyond simple time-series depictions, such as line charts and bar diagrams, using 2D, 3D, immersive environments, and relying on various metaphors and principled approaches [6]–[12]. However, many approaches oscillate between depicting details and providing the big picture, often in a trade-off of one at the expense of the other.

Given the sheer amount of information and the length of the time periods modern software evolution analysis is concerned with, it is clear that linear representations are suboptimal.

We present a novel approach that maps time onto a spiral, displaying the complete commit and branching history in full detail, yet in a scalable way. Already applied successfully for long time-series data [13], spiral layouts avoid the expansion of the time axis along only one direction and roll it up, compressing and optimizing space usage, but also highlighting recurrent patterns. For example, Figure 1 depicts 7 years of the history of a project named Roassal.² Each revolution lasts one year. All commits and the branches they belong to are mapped onto the spiral arms. Branches are differentiated by color. The cyclic nature of the spiral arms allows for comparison across time spans, in this case years. Among the patterns, we see how Roassal was intensely developed during the first 4 years, with development petering out in the second half of its lifetime, with very long-lived branches (some of them lasting years) but with relatively rare commits.

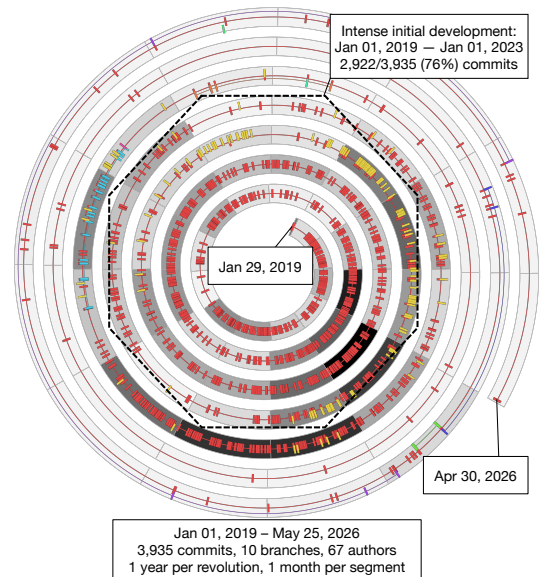


Fig. 1. Spiral view of the Roassal3 project repository.

We implemented our approach in NAUTILUS, an interactive web-based tool which we use to illustrate properties and emerging evolutionary patterns of human and agentic development processes on a set of case study repositories.

¹<https://github.com/torvalds/linux>

²<https://github.com/pharo-graphics/Roassal>

II. STATE OF THE ART

Software visualization research is concerned with easing the understanding of large software systems, often by decomposing large systems or aggregating insights from code-level inspection of software [14] supporting both the bottom-up and top-down comprehension of programs [15], [16]. When dealing with software evolution visualization, in particular, the additional constraint of representing the time dimension becomes central. The question is how to map time, and one initial classification of the proposed approaches can be: Those mapping time on time, and those mapping time on space.

Time on time: For mapping time on time, representing the passing of time through animation, Beyer and Hassan proposed animation storyboards, displaying sequences of depictions of structural software changes [17]. Armenti *et al.* leveraged interactive animations for commit-level analysis of file changes over time [18]. An alternative mapping of time on time is sonification. North *et al.* applied sonification techniques to the complete Git graph, including all commits and branches [11]. Armenti *et al.* “translated” activity levels, spikes, and individual developer contributions into different sounds [19].

Time on space: For mapping time on space instead, beyond the classical line charts where time flows from left to right, space can be used in clever ways to depict evolution. Serbout *et al.* leveraged scatterplots of the evolution of artifacts at each related commit timestamp [20]. The “RepoGrams” proposed by Rozenberg *et al.* similarly map user-defined metrics onto a horizontal condensed timeline of commits [10]. Kim *et al.* mined and mapped Git metadata to horizontal timelines [21]. Hermann *et al.* created a tabular layout for feature evolution, with calendar dates as rows and names of software features as columns, highlighting the resulting changes depending on the occurrence and type of change in the cells [22]. Kula *et al.* used line charts to visualize library dependency changes [23]. D’Ambros and Lanza presented a timeline approach to explore the relationships between software revisions and bugs [7].

Levels of granularity: The strategy for the mapping of time, although important, is just one possible classification parameter. Previous research proposed visualizations at different levels of granularity, ranging from system-wide representations down to individual files and methods. Taylor and Munro proposed the “Revision Towers” to reflect file changes and responsible authors [24]. Other file-level approaches include the “Evolution Spectrographs”, reflecting file changes by reducing color shades for each file, the less changes have occurred recently [25]. Girba *et al.* focused on visualizing software evolution through file authorship [26]. Fiechter *et al.* focused on GitHub issues and how they evolve over time [27]. The most detailed approaches include code-level analysis: The “history slicing” proposed by Servant and Jones is creating semantically meaningful spatio-temporal slices of code changes [28]. Hermann *et al.* aimed for a semantic understanding of software features mapped to concrete code changes [22]. Feist *et al.* semantically enriched the number of line-level changes with that in the abstract syntax tree [29].

Lanza visualized the evolution of classes’ code size, by mapping metrics like method and instance variable counts to heights and width of boxes, representing classes and putting them into a matrix, comparing those boxes across software versions [6]. Such class matrices are enriched by performance metrics in the work of Sandoval *et al.* [30]. Alexandru *et al.* presented “Evo-Clocks” mapping class-related metrics into circular layouts [12]. The baseline is that there is no approach to “rule them all”, and each method consciously focuses on specific aspects prioritizing the task or the domain at hand.

Multi-layer and aggregated: Honda *et al.* proposed a multi-layer approach for the visualization of code clones, displaying a bar chart overview of clone set sizes over time, lists pointing to changing clone sets between points and time, and direct pointers to the source code [31]. Code changes can be aggregated: Collberg *et al.* clustered and connected in a sequence of graphs, subsequent snapshots different of system aspects, like inheritance, control flow, and call graphs, decomposable up to single evolution time steps [32].

Approaches can also be categorized into 2D, 3D, and immersive (augmented and virtual reality—AR/VR). Examples include code cities by Wettel and Lanza [9], [33], the extension for collaborative exploration [34], and as an immersive VR visualization [35]–[37]. Ardigò *et al.* enriched the city metaphor with depictions of file types, metric values, and a time-dependent layout in a growing city [38]. Freire-Pozo *et al.* used AR to visualize dependency graphs between files [39].

Timelines, treemaps, and others: Most approaches mostly map time either by animation or onto a spatial axis, as in timeline charts. There are alternative layout approaches such as “Git-Truck”, to analyze authorship, change, and activity history through Git commits: Time is mapped on color shades and entities are laid out with rectangle or circle packing algorithms [40]. The “Evolution Radar” by D’Ambros *et al.* leveraged a circular layout to arrange logically coupled software modules throughout their evolution [41]. Elsen leveraged sunburst diagrams to display folder structures evolution across Git branches [42]. Volpe and Parra propose a radial tree layout, with nodes representing subsystems, colored depending on the amount of commits affecting this subsystem [4].

Spirals: Occhipinti *et al.* visualized file version evolution as 3D towers in a spiral pattern for ultra-scale Git repositories [43]. Previous literature also showed how spirals highlight periodical and cyclical patterns [44]–[47]. The approaches most related to ours are by Gu and Hübschmann [13] and by Tominski and Schumann [48]. Both works use spirals for their compact display of long time series data suitable for different domains such as genetics, biology, climate, and software repository data [13].

Inspired by these works, we base our approach on the idea of mapping time on an Archimedean spiral and apply it to Git repositories of arbitrary sizes, with user-defined and calendar-based time scales (*e.g.*, years, months, weeks, days), commit-level granularity, all branches, authorship information, branching and merging activity, and interactive analytics.

III. CONSTRUCTING SPIRALS FROM REPOSITORIES

The mathematical foundation behind our approach for depicting repositories is the Archimedean spiral. In Figure 2, we show the spiral for the activity in an example repository between 2024 and 2026. Branches are drawn as arcs along the spiral arm and commits, represented as rectangles, are positioned on top of these arcs. Colors represent branch membership. There are 8 commits on the “main” branch (red rectangles on the red line), 3 commits on first feature branch (blue), and 3 commits on a second feature branch (green).

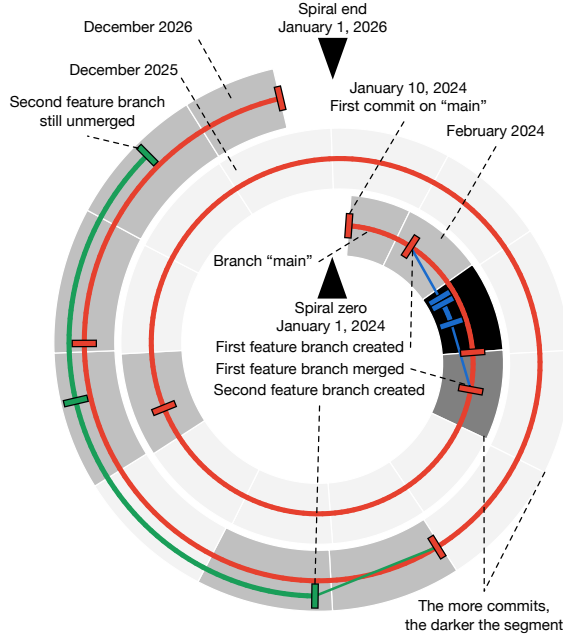


Fig. 2. Schematic view of an example repository spiral.

In this example, each spiral revolution represents one year and each segment of the spiral arm is one month. The geometric spiral start (referred to as “spiral zero” in the following) defaults to January 1 of the year of the first commit, useful for aligning spirals of multiple repositories for comparison. To indicate the beginning and end of activity in the repository, segments are drawn starting from the first and ending at the last commit’s timestamp, respectively.

A. Mapping Time onto a Spiral Layout

Archimedean spiral layouts have already been applied to other domains [13]. The repositories lifetime is mapped onto the spiral turning angle θ (in radians):

$$\theta = 2\pi * \frac{\Delta Seconds}{RevolutionSeconds}$$

where $\Delta Seconds$ is the number of seconds elapsed from the spiral zero and $RevolutionSeconds$ is the number of seconds in a revolution (e.g., seconds in one year in Figure 2). Following the example, in a yearly spiral, a commit c_1 at 12AM on January 1, 2024 has $\theta = 0$, whereas a commit c_2 at 12AM on January 1, 2025, exactly one year later, has $\theta = 2\pi$.

The resulting θ is then used to create the Archimedean spiral, by computing the spiral radius $r = a + b\theta$, where a is the minimum starting radius, and b is the growth factor. In our case both a and b are linearly proportional to the number of visible branches, to avoid overlaps and central clutter. As a result, c_1 is directly above c_2 , one spiral revolution higher.

In spirals, time can either be mapped onto the length of the spiral arc or onto the turning angle θ . We prefer the latter to make recurring patterns at the selected time scales emerge naturally also on the axis exiting from the spiral center, for example, yearly differences in the same months.

B. Background Segments

The background segments subdivide a revolution and can be configured to represent months, weeks, days, hours, minutes or seconds and are calendar-correct. For example, if one segment corresponds to a month, January will be long enough to encompass 31 days, February 28 days (or 29 in a leap year), and so forth. This can create a minimal shift for leap years due to the extra day, especially in repositories with a long lifespan, but it avoids the alternative of increasingly wide gaps between fixed-size background segments in the outer spiral revolutions.

The background segments have a configurable color (black by default), with their position following the aforementioned time mapping and a transparency proportional to the segment’s commit velocity (commits in the segment per segment time unit). By default, the maximum opacity is assigned to the segment with the maximum amount of commits, but it can be customized to any positive value (e.g., to clip outliers).

C. Branches

Branches are drawn as arcs between the first commit and the last (branch HEAD, in git lingo), following the spiral arm. Within the arm, branches are assigned to *lanes*: The default branch is assigned to the center lane, the remaining ones are sorted by ascending first commit’s timestamp and receive a lane, alternating between left and right of the default branch.

We generate branch colors in the hue, saturation, and value (HSV) color space, varying only the hue H with fixed saturation and value. To distribute hues deterministically for an arbitrary number of branches, we use a golden-angle progression.³ For a branch with identifier i :

$$H_i = (i \cdot \gamma) \bmod 360^\circ, \gamma = \frac{360^\circ}{\varphi^2} \approx 137.5^\circ, \varphi = \frac{1 + \sqrt{5}}{2}$$

The resulting HSV color is $C_i^{HSV} = (H_i, 0.70, 0.90)$

The head branch has $i = 0$, $H = 0^\circ$, and is therefore consistently of a red color. The $S = 0.70$ and $V = 0.90$ are kept constant to avoid oversaturated and excessively dark colors. Compared with rotating through a fixed-size palette, this approach provides a larger and deterministic set of branch colors. However, because the colors vary only along the hue dimension, large numbers of branches can still produce colors that are difficult to distinguish.

³<https://tinyurl.com/4myu3brd>

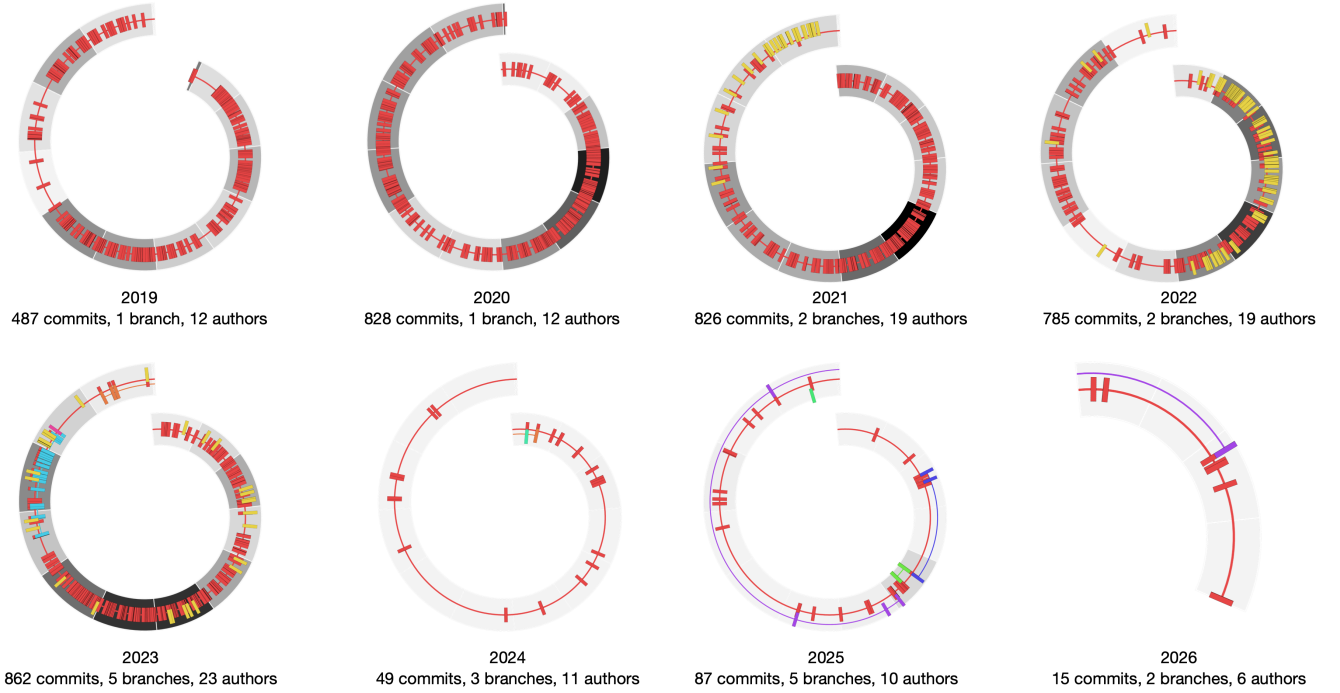


Fig. 3. Evolution of the *Roassal3* repository from 2019 to 2026.

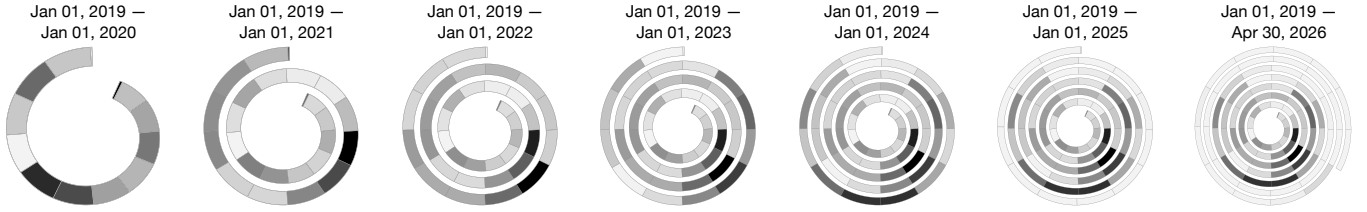


Fig. 4. Incremental evolution of the *Roassal3* repository from January 1, 2019 (background segments only).

D. Commit Markers

The commit markers are designed as rectangles positioned orthogonally to the spiral arm, on top of the line of their assigned branch. This slim design is space-efficient and reduces the overlaps with neighboring commits on the same lane, but also avoids complete overlaps between commits in different lanes. The commit markers share the color of their branch.

E. Inter-Branch Edges

We detect commits sharing a parent or a child in the Git graph to display branching and merging events (e.g., blue branch in Figure 2). If a commit has more than one child commit, for each child, we create a *branching edge* with the same color as the child commit on the other branch. Similarly, for *merging edges*, each parent of a commit that does not belong to the same branch gets a *merging edge* in the same color as the parent commit. Figure 2 shows a possible exception with the green branch, created in July 2025 but not merged back into main yet (the head can still be retrieved).

Branching and merging edges are not drawn as arcs, but as direct lines between the commits, reducing potential confusion with the regular branch lines and emphasizing large time gaps in the workflow (cross-arm-cutting edges).

F. Defining the Spiral Timespan

The default time window is the full repository life span and can be customized to arbitrary dates between the spiral zero and the date of the mining. This allows for detailed analyses at different granularities, from overviews to hourly evolution of a software system.

In Figure 3, we decompose the yearly evolution of *Roassal3*. After an initial ramp up phase, the repository sees an increased activity that becomes steady from 2020 to 2023. In 2019, August is clearly the most intense segment, with 92 commits. While the first five years are characterized by frequent activity, in the last quarter of 2023 and continuing until 2026, there is a sharp drop in activity, also displayed by the fading opacity of the background segments.

However, as we show in the progression in Figure 4, August 2019 is not the most active month overall: Its 92 commits are overshadowed by April 2020, “taking” the maximum opacity with its 178 commits, then May 2021 with the global maximum of 208. There is a progressive shift of the most active segments towards outer revolutions, revealed by moving the time window end forward and “animating” the spiral.

G. Different Time Scales and Analysis of Periodical Patterns

Figures 3 and 4 showcase the ability to compare timespans across revolutions: Years can easily be compared side by side or across spiral revolutions, even at finer granularities, such as individual monthly segments. But there are many other scales at which potential patterns can emerge. An example of how a spiral layout with flexible angular time mapping can reveal periodical patterns is the following: Weekends are expected to be periods of low activity for professional developers. However, spotting such small gaps within a yearly scale is difficult and requires inspection of patterns along the spiral. Our approach allows for different granularities of spiral revolution and segmentation. For example, Figure 5 shows development activity in *Roassal3*, from 2019 to 2022, with weekly revolutions and daily segmentation. The segments in the spiral slices corresponding to Saturday and Sunday appear usually less dark than on weekdays, such as Tuesday. At a glance, it becomes visible that weekends are the least active periods, consistently across several years. Without any commit-level inspection, from the shades of the background segments alone, spikes and recesses in periodical activity are naturally highlighted.

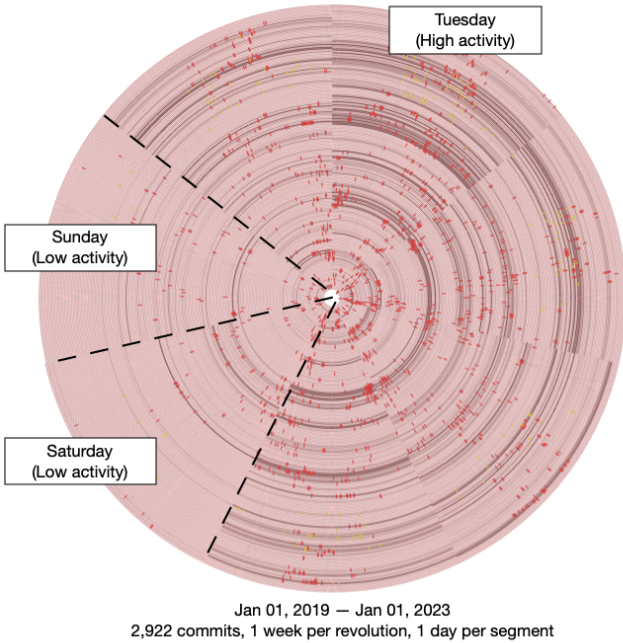


Fig. 5. Evolution of the *Roassal3* repository from 2019 to 2022 (weekly revolutions and daily segmentation).

IV. TOOL SUPPORT

We briefly detail NAUTILUS, the tool we built to implement our approach. NAUTILUS allows the user to generate spirals with different revolution and segmentation time granularities, time sub-windows or moving the spiral zero. Through its web-based user interface (UI), the user can interact with commits, branches, and segments, analyzing them in detail with additional information (*e.g.*, timestamp, commit message, author, segment commit velocity).

NAUTILUS opens as a web page in the browser, deployed at <https://nautilus.si.usi.ch>. The UI consists of the following elements: A large canvas displaying the spirals (Figure 6, C); a sidebar to select and configure locally-stored test repositories (R1) or mine new ones on demand (R2); a drawer opening on the right pane when the user double-clicks a commit, a background segment or an inter-branch edge, which shows interactive analytics (A). The central control element (bottom) is composed of the time-related settings and allows the user to configure the beginning (SS), the end (SE), the zero (Z), periodicity, and granularity (G) of the rendered spiral.

A. Repository Selection and Branch Management

NAUTILUS features sample repositories that can be selected from a menu (R1). It also offers the possibility to add/remove public GitHub repositories (R2). Once a test repository or a mined one is selected, in (B) the user can select the branches of that repository that should be rendered in the spiral on the canvas and if inter-branch edges should be displayed. Branches can be filtered based on their names, minimum and maximum lifetime, and commit counts.

B. General Spiral Settings

Depending on the number of branches the user selects, the spiral background can become very thick and dominate the visualization, making the commit markers and branch lines less visible. To mitigate this, the menu in (S) allows to change visual parameters (*e.g.*, increase the size of commit markers, increase the thickness of branch lines) to enhance visibility of specific elements. The same menu allows to change color and maximum opacity of the background segments.

C. Overview Metrics

The header in (H) shows statistics and overview information (*e.g.*, number of currently displayed commits, branches with “active” commits, open branches with no commits in the displayed time window (“alive”), and the number of authors that performed them.

D. Interactive Time settings

NAUTILUS contains controls to define revolution and segmentation time units (G), spiral start (SS) and spiral end (SE), using date and time pickers, quick-shift buttons for one-day steps and a two-sided time slider and with (Z), the spiral zero can be synchronized with the spiral start inputs. Available units in (G) are year, month, week, day, hour, minute, second.

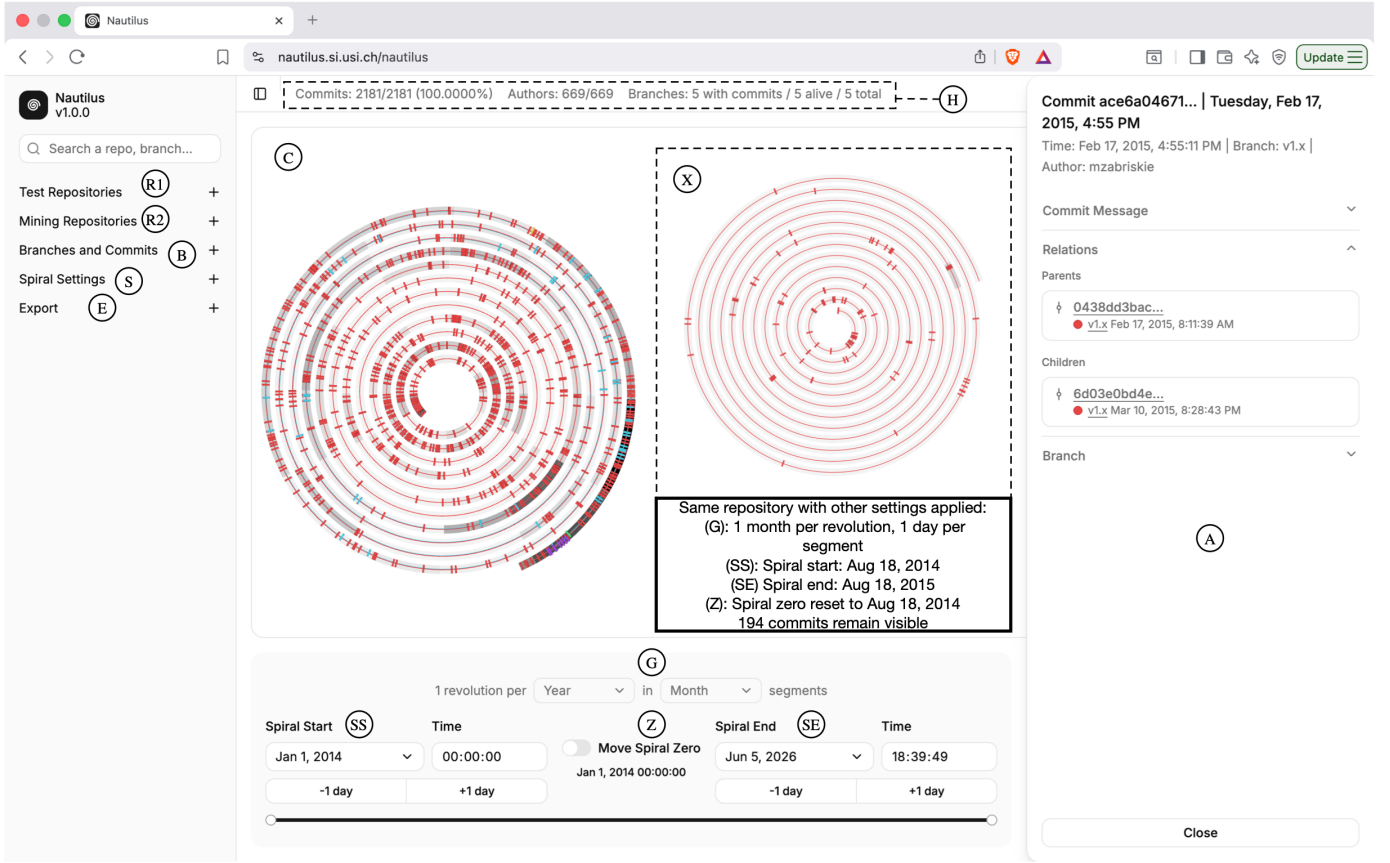


Fig. 6. UI of NAUTILUS.

The segmentation unit is always at least one level smaller than the revolution unit. The UI in Figure 6 shows the *Axios* repository with the selected time settings. It shows the complete 12-year life span of the repository, and its 2,181 commits over 5 branches. With the spiral zero being fixed to the default, Jan 1, the spiral starts with a circular offset from the 12 o'clock position of Jan 1, since the repository's first commit happens on Aug 18, 2014. In an alternative representation (X) the spiral spans one year from first commit, the switch to reset the spiral zero (Z) is activated, eliminating the offset, the spiral is configured to display one revolution per month, segmented in days, and results in 13 spiral revolutions.

E. Interactive Analytics

Every visual element in NAUTILUS is interactive.

Background segments offer single- and double-click actions. A single-click on a segment opens a tooltip displaying the represented time span, the number of commits, the average number of commits per hour, and the three most active branches and authors, in the selected segment. A double-click on a background segment opens a sidebar with: Commits which can be inspected through a direct link to GitHub, revealing code changes and tracing back to the original repository host; branches with commits in the selected time segment.

Clicking on a branch name in the list links to GitHub, automatically filtering for the commits assigned to this branch in the segment's time window; A list of authors committing changes during the selected time segment.

Commits can be hovered to highlight their relations (*e.g.*, parents, children) and inspected using a double-click: When a commit marker is hovered, all its parents, children and attached inter-branch edges are highlighted. The double click opens up a sidebar with a direct link to the commit on GitHub, displaying the branch, author, exact timestamp and commit message. Further details on the assigned branch can be displayed, including the resolved first and target commits, the count, date, and time range between the branches first and last included commit timestamps. The menu also contains a list of direct parent and child commits (A). All the commits and branch names link to GitHub.

Branches can be visually highlighted by single-clicking the branch lines, or, since commits can be very close to each other, also by single-clicking commits. Commits that do not belong to the clicked branch are greyed out to highlight the selection.

Inter-branch edges can be highlighted with a single-click action, emphasizing the endpoints of this edge, or can be double-clicked to open the analytics drawer, indicating whether this is a branching or merging event and showing the start and end commits of this edge, their parents, and their children.

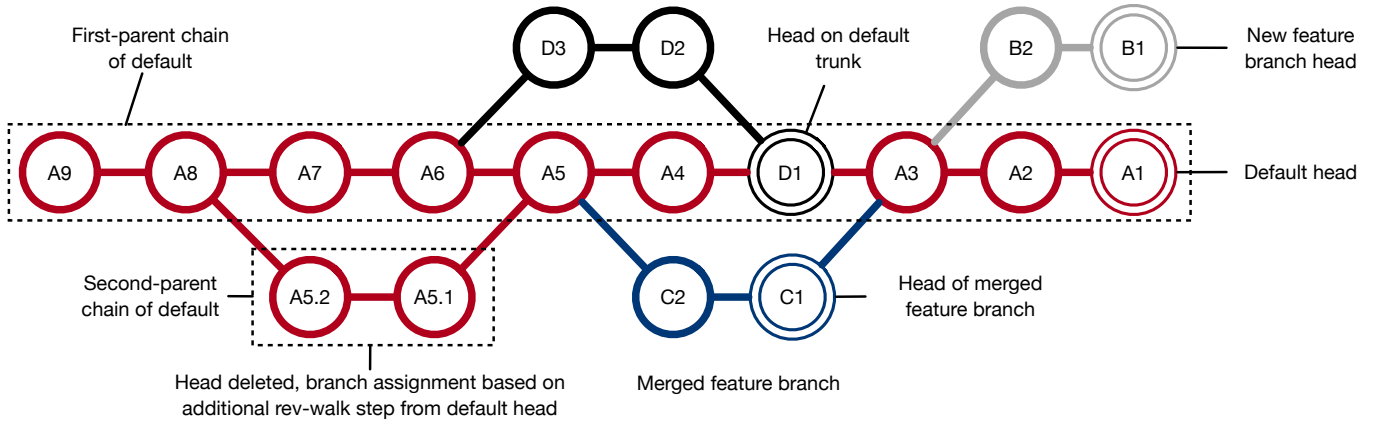


Fig. 7. A sample commit chain showcasing all different branching situations covered by NAUTILUS. Time goes from left to right, traversal is backwards starting from the heads on the right, first with the default branch. Branch assignment is color-coded.

F. GitHub Mining

NAUTILUS extracts branching, commit, and author information from Git repositories downloaded from GitHub. Commits and the corresponding author information are stored explicitly and can be read from the repository directly. The branch on which a commit was originally performed, however, can only be reconstructed accurately using the private Git log of the contributor [5]. Figure 7 shows a color-coded example of how our approach can reconstruct a typical git workflow.

Git keeps branch heads, the last commit in the commits chain of a branch. Starting from a head, it is possible to traverse the commit chain backwards through the commits’ parents and reconstruct the branch. To avoid being sidetracked by other branches that are merging into the current branch during traversal, only the first-parent chain must be traversed. This excludes commits merging into the current branch. Based on this knowledge, NAUTILUS is creating one possible history under the following three assumptions.

1. A branch contains at least all commits that are reachable from its head’s first-parent chain. NAUTILUS extracts each head pointer of the repository to mine and traverses its first-parent chain of commits. All the retrieved commits are assigned to the currently traversed branch. NAUTILUS stops when it reaches another branch head or a commit already assigned to another branch. It starts with the default branch, often stored as the “main” or “master” branch, so that its commit chain is guaranteed to be completely reconstructed.

2. The current default branch is the only default branch during the entire repository lifetime. To reconstruct the default branch, NAUTILUS takes the current default branch of the repository and reverse-walks the first-parent chain of the commits until the first commit of the repository. However, the default branch may have changed in the past.

3. Branch lifetime is defined by the first commit’s timestamp to the head’s timestamp. Since Git history is rewritable (e.g., by rebase operations [5]) this can cause commits to appear “detached” from the reconstructed branching history.

V. CASE STUDIES

We apply our approach to selected repositories. All were mined on May 6, 2026, except OpenClaw and Axios, mined on June 5, 2026. Axios is used for illustration in Section IV, while we updated OpenClaw to account for its extremely rapid evolution. Table I summarizes repository statistics.

Besides Axios and Roassal3, presented as examples in the previous sections, we have the *joynr* project, a web-based communication framework for distributed services. Its 13,188 commits, 54 active branches, and 120 contributors, under the guidance of the company owning the repository, BMW Car IT, serve as a representative for enterprise software development.

The Linux Kernel is one of the most popular and largest software repositories consisting of over 1.4 million commits in one single branch, starting from April 16, 2005 to date. The discrepancy between the development starting date and the creation of GitHub can be explained by the fact that Git, the backbone of GitHub, already existed before GitHub, yet even this first commit “forgets” the previous history of the project and is a fresh start for versioning in Git.

Node.js is a popular JavaScript runtime environment, created in February 2009 and still under active development. It consists of 102,003 commits across 64 branches and its development is driven by a large open-source community.

OpenClaw is a personal agentic AI assistant which can be deployed as a self-hosted instance and accessed via different communication channels. Its repository accumulated 70,965 commits across 2,306 branches in roughly 6 months of lifespan, driven by 2,799 authors, including an undefined share of AI agents pushing the development at hundreds of commits per day. According to its creator, the basis of *OpenClaw* and its predecessor were generated (“vibe coded” [49], [50]) using agentic AI systems.⁴

We will use these repositories to display exemplary cases of large-scale and fine-grained patterns emerging in software evolution histories and reflect on the results.

⁴<https://www.youtube.com/watch?v=7rzYDM6vMtI>

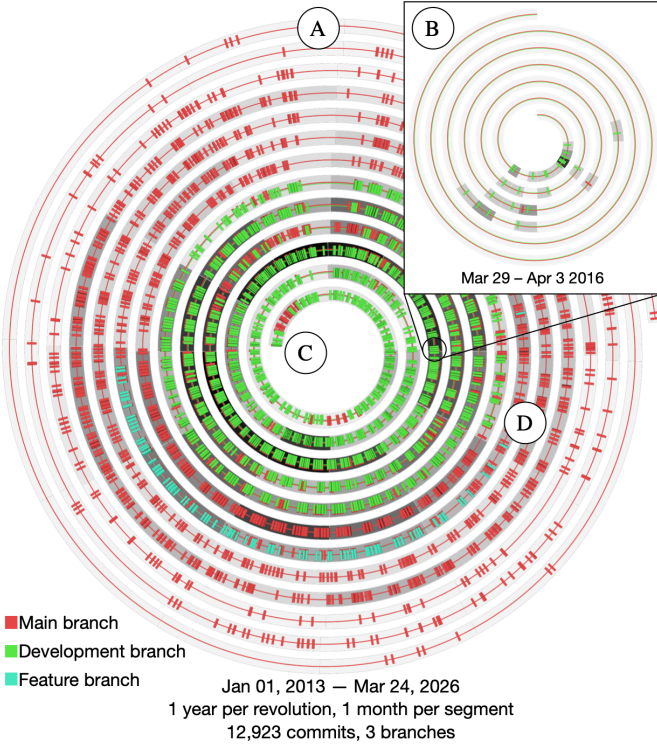
TABLE I

DESCRIPTIVE STATISTICS OF THE GIT REPOSITORIES UNDER ANALYSIS. VELOCITY IS REPORTED AS COMMITS PER DAY [C/D] ON THE DEFAULT BRANCH.

Repository	Activity Span	Commits	Branches	Authors	Velocity [c/d]	GitHub URL
Axios	Aug 18, 2014 – Jun 05, 2026	2,181	5	669	0.48	https://github.com/axios/axios
joynr	Oct 08, 2013 – Mar 24, 2026	13,188	54	120	1.68	https://github.com/bmwcarit/joynr
Linux Kernel	Jan 06, 2005 – May 06, 2026	1,444,589	1	42,593	34.32	https://github.com/torvalds/linux
Node.js	Feb 16, 2009 – May 06, 2026	102,003	64	4,963	7.20	https://github.com/nodejs/node
OpenClaw	Nov 24, 2025 – Jun 05, 2026	70,965	2,306	2,799	293.52	https://github.com/openclaw/openclaw
Roassal3	Jan 29, 2019 – Apr 30, 2026	3,935	10	67	1.44	https://github.com/pharo-graphics/Roassal

A. The Corporate Worker

Companies follow schedules: Contributions span from Monday to Friday with free weekends and visible holiday periods (e.g., the Christmas break). This is reflected in the yearly revolutions of the spiral of *joynr* (Figure 8). Commits are arranged in small blocks and unevenly spread around the spiral arm. An inspection with NAUTILUS’ tooltips reveals that the blank spaces between the commit “blocks” are in fact weekends. The Christmas break is visible in the top slice (A).

Fig. 8. Spiral of the *joynr* project repository, 3 branches.

A more detailed drill-down with daily revolutions and hourly background segments (B) reveals the working hours pattern: The late evening and night hours contain no commits for the project whose workforce is based in Ulm, Germany (one or two hours ahead of the UTC reference used by NAUTILUS), with activity consequently starting in the morning, while the two empty revolutions of the spiral are attributable to the weekend.

Our approach also makes the branching strategy pattern in this repository rather obvious: The main branch is collecting and carrying the final results, while a development branch is merged into the main one regularly. Both the main and the development branches are starting together at the beginning of the repository history (C). This branching strategy is then abandoned for single release branches that are appearing and disappearing for shorter periods of time (D). For example, the turquoise feature branch is living for about five months. In many cases, for those feature branches only the head commit remains, which indicates that most of the changes in the original branch are collapsed into one commit, referred to as “squashing” in Git terminology.

B. The Globetrotter

Although Node.js is also under corporate governance, it does not show the same working schedule of the *joynr* project. A spiral with daily revolutions and hourly segments reveals a uniform spread of activity across the day, with the early hours of the UTC timezone being slightly less active (Figure 9).

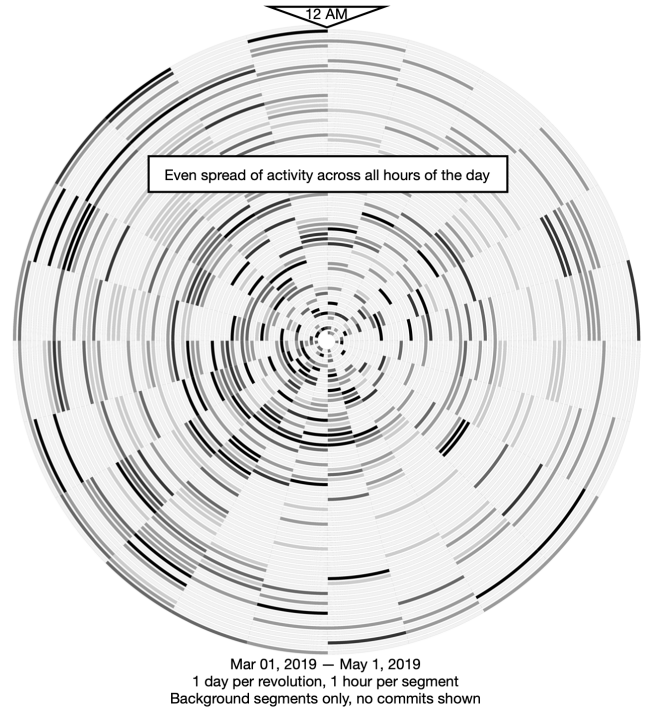


Fig. 9. Spiral of the Node.js repository (no commits shown).

This time pattern can be explained with a broad spread of the many community members around the world, considering that in February 2015, there were already 27 translation and localization teams involved in the development of Node.js. However, before it turned into this stable large scale open-source project, whose complete lifespan is shown in Figure 10, Node.js has experienced organizational difficulties.

Unsatisfied by the governance of the project, which was supervised by the company Joyent, a small subset of Node.js developers decided to fork the project and create a parallel version, *io.js*. Quickly gaining traction and advancing development faster than the origin of the fork, *io.js* convinced Joyent to try and reunite with its former community members. This ended with the reunification in June 15, including merging the two code bases. In 2016, the development notably accelerated, as reflected by the darker shades of the background segments of the spiral (Figure 10 enlarged focus) compared to those between the “revolution” of 2009 and “reunification” of 2015, a pace still steady today.

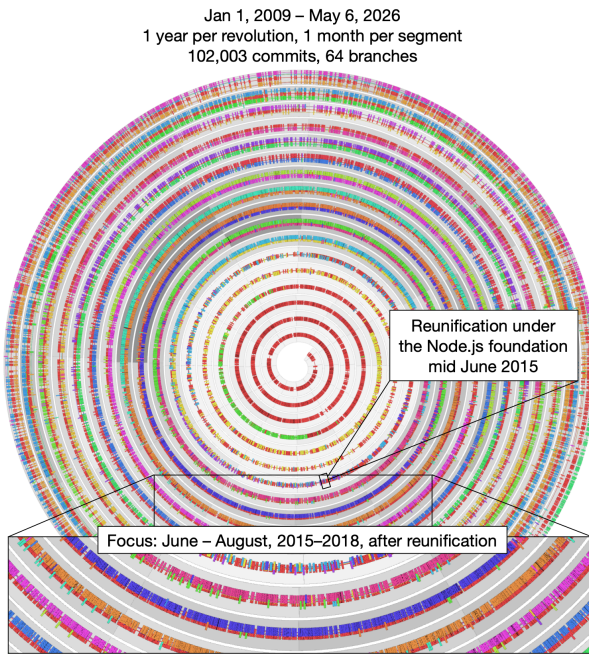


Fig. 10. Spiral of the Node.js repository. All branches and commits.

C. The Pioneer

The next example of interest, on a different scale for longevity and community effort, is the Linux kernel, shown in Figure 11. The pioneer pattern shows a period of significant instability and unstructured development before 2008, while most of the commit activity in the outer revolutions is regular.

A quick research on the events between 2005 and 2008 concerning the development of the Linux kernel yields interesting insights: In 2005, the Linux project abandoned BitKeeper as a version control system in favor of the newly developed Git, conceived by Linus Torvalds to replace BitKeeper.

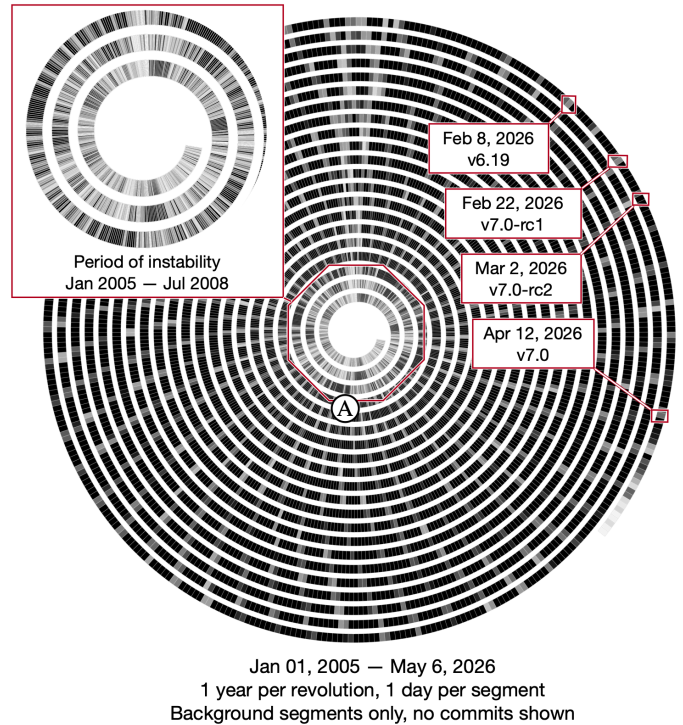


Fig. 11. Spiral of the Linux kernel repository.

In 2007, first complaints about the slow and error-prone integration of multiple kernel versions emerged. In February 2008, a new approach to handle patch integration in a new source-tree was proposed, keeping the main line of releases clean. In July 2008, this was introduced as the official integration practice. This event marks a change in the process (A), where the development of the Linux kernel becomes more structured and aligned to a periodical release pattern.

This periodical release pattern also emerges from smaller substructures: There are weekly activity drops visible in Figure 11 such as bi-weekly and even after longer periods. Those can be attributed to the specific release schedule the Linux kernel is following:⁵ At the beginning of each development cycle, there is a merge window lasting approximately two weeks, which is the time to integrate stable, reviewed code changes into the mainline kernel. Then, a phase of patching begins, in which Linus Torvalds releases a more stable kernel each week. Those activity patterns are also reflected in the spiral of the Linux repository. One example of this can be seen in February 2026 in the outermost revolution of the spiral in Figure 11. On February 8, the definitive version of the Kernel 6.19 is released. After two weeks, the timespan of the merging window, the first patched version, named “7.0-rc1”, is released. This weekly pattern repeats until the final new version, v7.0 is released on April 12, 2026.

D. The Machine

The OpenClaw repository shows AI-driven software development in action. According to Steinberger, its initial developer, at least the first version of OpenClaw was an extended

⁵<https://tinyurl.com/5n6fr2h9>

WhatsApp bot entirely developed using AI agents. Figure 12 shows how this development trend continues on a hard to grasp magnitude: Starting from the end of November 2025 the repository accumulates 6,045 commits (8.52%) in 48 branches in one month and a half. Then agentic development kicks in, producing 64,920 commits (91.48%) across 2,258 branches in the following 5 months, speeding up the contribution pace by roughly 4 times. The lifespan of a single branch is extremely short, merged into main very quickly.

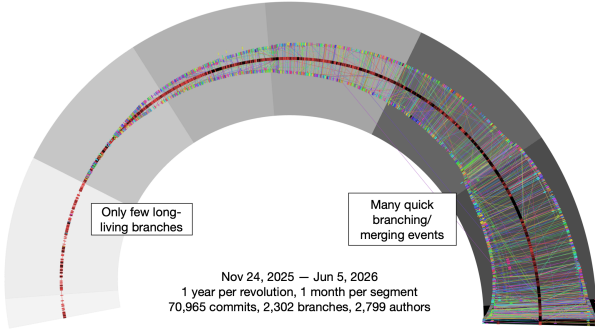


Fig. 12. Spiral of the OpenClaw repository.

Already at a large scale, the rapid development speed and integration of parallel branches is astonishing: While the Linux kernel is concerned with a consistent, integrated, and convergent mainline of development, the maintainers of OpenClaw prioritize fast integration, almost instantly merging branches after their creation.

Time is also not an issue for AI-driven development: In May 2026, segmented in hourly steps, Figure 13 shows how there is hardly any time of the day where the repository is not undergoing some form of maintenance or evolution, or where branches are not created and merged. With an average of 28.17 commits per hour (approximately one commit every two minutes), the pace is sustained throughout the entire month. Peter Steinberger “alone” contributed 10,211 commits in that time frame. Although the commits contain author names, maintaining such a high commit velocity over an extended timespan is unrealistic for a human to achieve, especially considering the fact that many authors contribute under more than one alias [51]. In contrast, the most active month in Node.js, October 2017, has an average velocity of 2.16 commits per hour, and this is mainly achieved in one single day, October 6, with 25.46 commits per hour.

VI. DISCUSSION AND LIMITATIONS

Our approach uses a consistent and flexible way to visualize periodical data for comparison of timespans, particularly suitable for software repositories and their evolution processes. One limitation is the misrepresentation of time with visual errors that increase towards the outer revolutions of the spiral, where commit markers are more spaced compared to the inner ones.

This may distort time perception in recent activity. To mitigate this, the saturation of the background segments, based

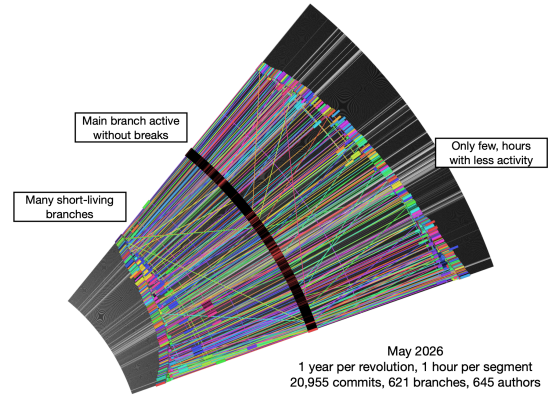


Fig. 13. OpenClaw development in May 2026.

on the hourly commit rate, gives a visual undistorted cue about commit velocity at the selected segment granularity of interest.

While our approach constructs a valid and consistent history for any Git repository, this is still based on heuristics to disambiguate conflicts that Git operations might introduce.

We chose a calendar-correct segmentation of the spiral arm to consistently align commit time stamps with actual calendar dates. This creates a slight shift between segments in leap years, setting off the spiral revolution by one day every leap year. The alternative is to use fixed-size background segments, but misrepresent time with an inconsistent placement of commit markers of recurrent events, masking temporal patterns.

In repositories such as the Linux kernel, the commit density can be so high that commit markers become indistinguishable. Our approach compensates for this with alternative temporal resolutions and sub-window selection, reducing visual clutter.

VII. CONCLUSION

We presented a scalable, yet details-preserving approach to software evolution visualization, mapping a software repositories lifetime onto a spiral. We start by recreating the history of a GitHub repository, extracting commits as the basic proxy of development contributions. NAUTILUS offers a rich toolkit to configure repository spirals to custom time windows with different granularities and also interactive analytics to investigate activity patterns more in depth. Laying this data out on a spiral, mapping time to the turning angle thus consciously trading linear distance precision for angular placement, surfaces interesting recurrent visual patterns in our case studies. Ranging from corporate IT and open-source projects to fast-paced AI-driven software repositories we show ample evidence of the usefulness of our approach for consistent analysis of long periodical time series in software maintenance and evolution.

Reproduction package: To ensure the verifiability of our work, data and high resolution images are available at <https://doi.org/10.6084/m9.figshare.32649417>

Acknowledgments: This work is supported by the Swiss National Science Foundation (SNSF) through the project “FORCE” (SNF Project No. 232141).

REFERENCES

- [1] L. A. Belady and M. Lehman, *Program evolution: Processes of software change*. Academic Press, 1985.
- [2] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “An in-depth study of the promises and perils of mining GitHub,” *Empirical Software Engineering*, vol. 21, no. 5, pp. 2035–2071, 2016.
- [3] M. Hansen, L. Damerau, D. König, and W. Hasselbring, “Visualization of the linux kernel with explorviz,” in *Proceedings of VISSOFT 2025 (Working Conference on Software Visualization)*. IEEE, 2025, pp. 117–120.
- [4] A. Volpe and E. Parra, “Visualizing software evolution in linux: A hierarchical graph-based heat map,” in *Proceedings of VISSOFT 2025 (Working Conference on Software Visualization)*. IEEE, 2025, pp. 129–132.
- [5] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, “The promises and perils of mining git,” in *2009 6th IEEE International Working Conference on Mining Software Repositories*. IEEE, 2009, pp. 1–10.
- [6] M. Lanza, “The evolution matrix: Recovering software evolution using software visualization techniques,” in *IWPSE '01: Proceedings of the 4th International Workshop on Principles of Software Evolution*. ACM, 2001, pp. 37–42.
- [7] M. D'Ambros and M. Lanza, “Software bugs and evolution: A visual approach to uncover their relationship,” in *Proceedings of CSMR 2006 (10th Conference on Software Maintenance and Reengineering)*. IEEE, 2006, pp. 10–pp.
- [8] F. Steinbrückner and C. Lewerentz, “Representing development history in software cities,” in *Proceedings of SOFTVIS 2010 (International Symposium on Software visualization)*, 2010, pp. 193–202.
- [9] R. Wettel, M. Lanza, and R. Robbes, “Software systems as cities: A controlled experiment,” in *Proceedings of ICSE 2011 (33rd International Conference on Software Engineering)*. ACM, 2011, pp. 551–560.
- [10] D. Rozenberg, I. Beschastnikh, F. Kosmale, V. Poser, H. Becker, M. Palyart, and G. C. Murphy, “Comparing repositories visually with repograms,” in *Proceedings of the 13th International Conference on Mining Software Repositories*. ACM, 2016, pp. 109–120.
- [11] K. J. North, A. Sarma, and M. B. Cohen, “Understanding git history: A multi-sense view,” in *Proceedings of the 8th International Workshop on Social Software Engineering*. ACM, 2016, pp. 1–7.
- [12] C. V. Alexandru, S. Proksch, P. Behnamghader, and H. C. Gall, “EvoClocks: Software evolution at a glance,” in *Proceedings of VISSOFT 2019 (Working Conference on Software Visualization)*. IEEE, 2019, pp. 12–22.
- [13] Z. Gu and D. Hübschmann, “spiralize: an R package for visualizing data on spirals,” *Bioinformatics*, vol. 38, no. 5, pp. 1434–1436, 2021.
- [14] S. G. Eick and T. Ball, “Software visualization in the large,” *Computer*, vol. 29, no. 04, pp. 33–43, 1996.
- [15] M.-A. D. Storey, F. Fracchia, and H. A. Müller, “Cognitive design elements to support the construction of a mental model during software visualization,” in *Proceedings Fifth International Workshop on Program Comprehension. IWPC'97*. IEEE, 1997, pp. 17–28.
- [16] M.-A. D. Storey, “Theories, methods and tools in program comprehension: Past, present and future,” in *13th International Workshop on Program Comprehension (IWPC'05)*. IEEE, 2005, pp. 181–191.
- [17] D. Beyer and A. E. Hassan, “Animated visualization of software history using evolution storyboards,” in *2006 13th Working Conference on Reverse Engineering*. IEEE, 2006, pp. 199–210.
- [18] C. Armenti and M. Lanza, “Using interactive animations to analyze fine-grained software evolution,” in *Proceedings of VISSOFT 2024 (Working Conference on Software Visualization)*. IEEE, 2024, pp. 36–47.
- [19] C. Armenti, M. Raglianti, and M. Lanza, “Sonifying and visualizing the heartbeat of evolving software systems,” in *Proceedings of VISSOFT 2025 (Working Conference on Software Visualization)*. IEEE, 2025, pp. 1–12.
- [20] S. Serbout, D. C. M. Hurtado, H. Atwi, E. Riggio, and C. Pautasso, “Evoscat: Exploring software change dynamics in large-scale historical datasets,” in *Proceedings of VISSOFT 2025 (Working Conference on Software Visualization)*. IEEE, 2025, pp. 94–105.
- [21] Y. Kim, J. Kim, H. Jeon, Y.-H. Kim, H. Song, B. Kim, and J. Seo, “Githru: Visual analytics for understanding software development history through git metadata analysis,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 656–666, 2021.
- [22] K. Hermann, J. Martinson, and T. Berger, “Visualizing Feature-Oriented software evolution,” in *Proceedings of the 29th ACM International Systems and Software Product Line Conference - Volume A*. ACM, 2025, pp. 136–141.
- [23] R. G. Kula, C. De Roover, D. German, T. Ishio, and K. Inoue, “Visualizing the evolution of systems and their library dependencies,” in *Proceedings of VISSOFT 2019 (Working Conference on Software Visualization)*. IEEE, 2014, pp. 127–136.
- [24] C. M. B. Taylor and M. Munro, “Revision towers,” in *Proceedings of VISSOFT 2002 (International Workshop on Visualizing Software for Understanding and Analysis)*. IEEE, 2002, pp. 43–50.
- [25] J. Wu, C. W. Spitzer, A. E. Hassan, and R. C. Holt, “Evolution spectrographs: Visualizing punctuated change in software evolution,” in *Proceedings. 7th International Workshop on Principles of Software Evolution*, 2004. IEEE, 2004, pp. 57–66.
- [26] T. Gırba, A. Kuhn, M. Seeberger, and S. Ducasse, “How developers drive software evolution,” in *Eighth International Workshop on Principles of Software Evolution (IWPSE'05)*. IEEE, 2006, pp. 113–122.
- [27] A. Fiechter, R. Minelli, C. Nagy, and M. Lanza, “Visualizing GitHub issues,” in *Proceedings of VISSOFT 2021 (Working Conference on Software Visualization)*. IEEE, 2021, pp. 155–159.
- [28] F. Servant and J. A. Jones, “History slicing: Assisting code-evolution tasks,” in *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*. ACM, 2012, pp. 1–11.
- [29] M. D. Feist, E. A. Santos, I. Watts, and A. Hindle, “Visualizing project evolution through abstract syntax tree analysis,” in *Proceedings of VISSOFT 2016 (Working Conference on Software Visualization)*. IEEE, 2016, pp. 11–20.
- [30] J. P. Sandoval Alcocer, F. Beck, and A. Bergel, “Performance evolution matrix: Visualizing performance variations along software versions,” in *Proceedings of VISSOFT 2019 (Working Conference on Software Visualization)*. IEEE, 2019, pp. 1–11.
- [31] H. Honda, S. Tokui, K. Yokoi, E. Choi, N. Yoshida, and K. Inoue, “CCEvoVis: A clone evolution visualization system for software maintenance,” in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 2019, pp. 122–125.
- [32] C. Collberg, S. Kobourov, J. Nagra, J. Pitts, and K. Wampler, “A system for graph-based visualization of the evolution of software,” in *Proceedings of the 2003 ACM symposium on Software visualization*. ACM, 2003, p. 77.
- [33] R. Wettel and M. Lanza, “Visualizing software systems as cities,” in *2007 4th IEEE International Workshop on Visualizing Software for Understanding and Analysis*. IEEE, 2007, pp. 92–99.
- [34] M. Hansen, H. Bielfeldt, A. Bernstetter, T. Kwasnitschka, and W. Hasselbring, “A software visualization approach for multiple visual output devices,” in *Proceedings of VISSOFT 2024 (Working Conference on Software Visualization)*. IEEE, 2024, pp. 101–105.
- [35] F. Fittkau, A. Krause, and W. Hasselbring, “Exploring software cities in virtual reality,” in *Proceedings of VISSOFT 2015 (Working Conference on Software Visualization)*. IEEE, 2015, pp. 130–134.
- [36] L. Merino, M. Ghafari, C. Anslow, and O. Nierstrasz, “Cityvr: Gameful software visualization,” in *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2017, pp. 633–637.
- [37] D. Moreno-Lumbreras, S. Montes-Leon, J. M. Gonzalez-Barahona, and G. Robles, “Immersion vs. familiarity: A controlled experiment of software evolution visualization in virtual reality,” *Empirical Software Engineering*, vol. 31, no. 4, 2026.
- [38] S. Ardigò, C. Nagy, R. Minelli, and M. Lanza, “M3triCity,” in *Proceedings of ICSE-C 2022 (International Conference on Software Engineering: Companion Proceedings)*. ACM, 2022, pp. 130–133.
- [39] D. Freire-Pozo, K. Cespedes-Arancibia, L. Merino, A. Fernandez-Blanco, A. Neyem, and J. P. Sandoval Alcocer, “Dgt-ar: Visualizing code dependencies in ar,” in *Proceedings of VISSOFT 2023 (Working Conference on Software Visualization)*, 2023, pp. 95–99.
- [40] K. Hojelse, T. Kilbak, J. Rossum, E. Japelt, L. Merino, and M. Lungu, “Git-truck: Hierarchy-oriented visualization of git repository evolution,” in *Proceedings of VISSOFT 2022 (Working Conference on Software Visualization)*. IEEE, 2022, pp. 131–140.
- [41] M. D'Ambros, M. Lanza, and M. Lungu, “The evolution radar: Visualizing integrated logical coupling information,” in *Proceedings of the 2006 international workshop on Mining software repositories*. ACM, 2006, pp. 26–32.

- [42] S. Elsen, “Visgi: Visualizing git branches,” in *Proceedings of VISSOFT 2013 (Working Conference on Software Visualization)*. IEEE, 2013, pp. 1–4.
- [43] G. Occhipinti, C. Nagy, R. Minelli, and M. Lanza, “Syn: Ultra-scale software evolution comprehension,” in *Proceedings of the 31st IEEE/ACM International Conference on Program Comprehension (ICPC 2023)*. IEEE, 2023, pp. 69–73.
- [44] J. V. Carlis and J. A. Konstan, “Interactive visualization of serial periodic data,” in *Proceedings of the 11th annual ACM symposium on User interface software and technology*. ACM, 1998, pp. 29–38.
- [45] M. Weber, M. Alexa, and W. Müller, “Visualizing time-series on spirals,” in *Proceedings of the IEEE Symposium on Information Visualization 2001 (INFOVIS’01)*. IEEE, 2001, p. 7.
- [46] M. O. Ward and B. N. Lipchak, “A visualization tool for exploratory analysis of cyclic multivariate data,” *Metrika*, vol. 51, no. 1, pp. 27–37, 2000.
- [47] K. P. Hewagamage, M. Hirakawa, and T. Ichikawa, “Interactive visualization of spatiotemporal patterns using spirals on a geographical map,” in *Proceedings 1999 IEEE Symposium on Visual Languages*. IEEE, 1999, pp. 296–303.
- [48] C. Tominski and H. Schumann, “Enhanced interactive spiral display,” in *Proceedings of SIGRAD (Annual SIGRAD Conference, Special Theme: Interactivity)*, no. 34. Linköping University Electronic Press, 2008, pp. 53–56.
- [49] Y.-H. Chou, B. Jiang, Y. W. Chen, M. Weng, V. Jackson, T. Zimmermann, and J. A. Jones, “Building software by rolling the dice: A qualitative study of vibe coding,” *arXiv preprint arXiv:2512.22418*, 2025.
- [50] A. Fawzy, A. Tahir, and K. Blincoe, “Vibe coding in practice: Motivations, challenges, and a future outlook—a grey literature review,” *arXiv preprint arXiv:2510.00328*, 2025.
- [51] T. Fry, T. Dey, A. Karnauch, and A. Mockus, “A dataset and an approach for identity resolution of 38 million author ids extracted from 2b git commits,” in *Proceedings of the 17th International Conference on Mining Software Repositories*. ACM, 2020, pp. 518–522.